

Nombre Alumno: Enzo Romero

Nombre Profesor: **Yanina Medina – Andrea Lezcano Airdi - Matías Mascazzini**

Grupo Laboratorio: Comisión 2.

DNI: 41948148.

TP: Trabajo Practico N°2.

Fecha de Entrega: Domingo 23/08/26.

Informe de Fundamentación Teórica y Meta-Análisis.

1) Introducción y objetivos:

El presente informe tiene como propósito fundamentar la aplicación de los conceptos teóricos del paradigma Orientado a Objetos desarrollados durante la resolución de la serie de ejercicios del TP N°2 a lo largo de la práctica se buscó aplicar los conceptos abstrayendo entidades del mundo real, modelando sus estados y comportamientos y garantizar un diseño de código mantenible, seguro y reutilizable.

2). Fundamentación de Conceptos Teóricos y Ejemplos de Aplicación

-Abstracción: Consiste en identificar las características y comportamientos esenciales de un objetivo del mundo real, descartando los detalles irrelevantes para el problema a resolver. En el ejercicio de la clase “Persona”, se abstrajeron únicamente las propiedades necesarias para la gestión de datos Básicos y el cálculo de la edad (nroDni, nombre, apellido, añoNacimiento), dejando de lado otros atributos no necesarios como la dirección teléfono o redes sociales etc.

- Encapsulamiento y ocultamiento de la información : Es el principio mediante el cual se restringe el acceso directo al estado interno de un objeto, protegiéndolo de modificaciones indebidas desde el exterior. Esto se logra definiendo los atributos de las clases modelo con el modificador de visibilidad “private”. Por ejemplo, en la clase Empleado, el atributo “ sueldoBasico ” es privada, impidiendo que en una clase externa altere directamente su monto sin pasar por la lógica del sistema .

```
4 public class Empleado{
5     private long cuil;
6     private String apellido;
7     private String nombre;
8     private double sueldoBasico;
9     private int añoIngreso;
10
11     // ...
12 }
```

-Doble Encapsulamiento y la Autorreferencia this: Esta técnica extiende el encapsulamiento al ámbito interno de la propia clase, forzando a que los métodos de la entidad no accedan de forma directa a las variables de instancia, sino a través de sus correspondientes métodos mutadores (Setters) y observadores (getters). En el método edad() de la clase persona, en lugar de operar directamente con el atributo añoNacimiento, se invoca la función observadora this.getAñoNacimiento() la palabra reservada this actúa como una referencia explícita al objeto actual que ejecuta el mensaje.

```
public int edad() {
    Calendar fechaHoy = new GregorianCalendar();
    int añoHoy = fechaHoy.get(Calendar.YEAR);
    return añoHoy - this.getAñoNacimiento();
}
```

-Sobrecarga de Constructores : la sobrecarga permite definir múltiples constructores para una misma clase, diferenciados por su firma (cantidad o tipo de parámetros). En la clase “Punto”, se implementaron dos constructores: uno por defecto punto(), que inicializa las coordenadas en el origen(0,0) y el otro explícito punto(p_x, p_y), que sitúa el objeto en coordenadas específicas provistas por el usuario.

```
10 // CONSTRUCTOR 1: Sin parámetros (sitúa el punto en el origen X=0, Y=0)
11 public Punto() {
12     this.setX(0.0);
13     this.setY(0.0);
14 }
15
16 // CONSTRUCTOR 2: Explícito con parámetros
17 public Punto(double p_x, double p_y) {
18     this.setX(p_x);
19     this.setY(p_y);
20 }
21
```

-Colaboración entre objetos e interacción con el ejecutable: los objetos no trabajan de forma aislada, sino que colaboran mediante el envío de mensajes. En las clases ejecutables (main), se instancian los objetos invocando sus métodos públicos (como mostrar(), desplazar(), o sueldoNeto()) respetando la interfaz pública de cada entidad.

Parámetros estáticos

```
9 public class PersonaTest{
10     public static void main (String []args){
11         Persona personal = new Persona(41948148, "Enzo", "Romero", 1999);
12         personal.mostrar();
13     }
14 }
```

parámetros por argumentos del main.

```

1 import java.util.Scanner;
2 /**
3  *clase ejecutable para probar la instanciacion de cliente
4  *utilizando la entrada de parametros desde el argumento del metodo main
5  *@autor Enzo Romero
6  *
7  */
8  public class ClienteTest{
9      public static void main (String []args){
10         if(args.length >=4){
11             int dni = Integer.parseInt(args[0]);
12             String apellido = args[1];
13             String nombre=args[2];
14             double saldo= Double.parseDouble(args[3]);
15             Cliente unCliente= new Cliente(dni,apellido,nombre,saldo);
16             unCliente.mostrar();
17         }else{
18             System.out.println("faltan argumentos");
19         }
20     }
21 }
22 }

```

Parametros pedidos por teclado .

```

1 import java.util.Scanner;
2
3 /**
4  * Clase ejecutable para probar la clase Punto mediante ingreso por teclado.
5  *
6  * @author Enzo Romero
7  */
8  public class EjecutablePunto {
9
10     public static void main(String[] args) {
11         Scanner teclado = new Scanner(System.in);
12
13         System.out.println("=== CREANDO PUNTO CON VALORES INICIALES ===");
14         System.out.print("Ingrese coordenada X: ");
15         double x = teclado.nextDouble();
16
17         System.out.print("Ingrese coordenada Y: ");
18         double y = teclado.nextDouble();
19
20         // Se instancia el punto usando el constructor con parámetros
21         Punto unPunto = new Punto(x, y);
22
23         System.out.println("\nEstado inicial del punto:");
24         unPunto.mostrar();
25         System.out.println("Coordenadas formateadas: " + unPunto.coordenadas());
26
27         System.out.println("\n=== DESPLAZANDO EL PUNTO ===");
28         System.out.print("Ingrese el desplazamiento en X (dx): ");
29         double dx = teclado.nextDouble();
30
31         System.out.print("Ingrese el desplazamiento en Y (dy): ");
32         double dy = teclado.nextDouble();
33
34         // Desplazamos el punto
35         unPunto.desplazar(dx, dy);
36
37         System.out.println("\nEstado del punto después del desplazamiento:");
38         unPunto.mostrar();
39         System.out.println("Nuevas coordenadas: " + unPunto.coordenadas());
40
41         teclado.close();

```

3) Conclusión: La resolución del TP2 permitió afianzar la correcta estructuración de código bajo el paradigma de objetos. La aplicación rigurosa de la abstracción y el doble encapsulamiento no solo garantiza la integridad de los datos, sino que facilita el mantenimiento futuro del software, permitiendo modificar las reglas de negocio internas sin impactar en las clases cliente o ejecutable.